Non-Interactive Black-Box WI Commit-and-Prove via Targeted Hitting-Set Generators

Susumu Kiyoshima

TCC 2026

© NTT, Inc. 2026

Starting point

Roei Tell's invited talk at TCC 2025

- Recent tools for derandomization: targeted PRGs and **targeted hitting-set generators (HSGs)**.
- They have cryptographic applications via **Fiat–Shamir in the plain model**.



© NTT, Inc. 2026

Starting point

Roei Tell's invited talk at TCC 2025

- Recent tools for derandomization: targeted PRGs and **targeted hitting-set generators (HSGs)**.
- They have cryptographic applications via **Fiat–Shamir in the plain model**.



This work

We use the connection between targeted HSGs and Fiat–Shamir to obtain a new cryptographic application.

© NTT, Inc. 2026

Targeted HSGs: motivation

Goal: deterministically emulate a one-sided-error algorithm $A(x;r)$.

$$x \in \mathsf{L}: \Pr_r[A(x;r)=1] \geq \tfrac{1}{2}, \qquad x \notin \mathsf{L}: A(x;r)=0 \text{ for every } r.$$

© NTT, Inc. 2026

Targeted HSGs: motivation

Goal: deterministically emulate a one-sided-error algorithm $A(x;r)$.

$$x \in \mathbf{L}: \ \Pr_r[A(x;r)=1] \geq \tfrac{1}{2}, \qquad x \notin \mathbf{L}: \ A(x;r)=0 \text{ for every } r.$$

Strategy:

1. Deterministically generate a list $S = \{r_1, \ldots, r_L\}$.
2. Run $A(x;r_1), \ldots, A(x;r_L)$, and accept iff any run accepts.

© NTT, Inc. 2026

Targeted HSGs: motivation

Goal: deterministically emulate a one-sided-error algorithm $A(x; r)$.

$$x \in \mathbf{L}: \ \Pr_r[A(x; r) = 1] \geq \tfrac{1}{2}, \qquad x \notin \mathbf{L}: \ A(x; r) = 0 \text{ for every } r.$$

Strategy:

1. Deterministically generate a list $S = \{r_1, \ldots, r_L\}$.
2. Run $A(x; r_1), \ldots, A(x; r_L)$, and accept iff any run accepts.

Requirement on S: Hitting condition for input x

$$\Pr_r[A(x; r) = 1] \geq \tfrac{1}{2} \implies \exists r_i \in S: \ A(x; r_i) = 1.$$

Efficiency goal: keep $|S|$ small to reduce the number of runs.

© NTT, Inc. 2026

Targeted HSGs: definition

NTT

A targeted HSG G for a class $\mathcal{C}$ and an input distribution $\mathcal{D}$

Syntax: $G(x)$ deterministically outputs a list $S := G(x)$.
Hitting guarantee: For each algorithm $A \in \mathcal{C}$, the list $S = G(x)$ satisfies the hitting condition for $A(x;\cdot)$ with high probability over $x \sim \mathcal{D}$.*

*While a standard HSG outputs a single list that satisfies the hitting condition for $A(x;\cdot)$ for every x, a targeted HSG may output a list that depends on x.

© NTT, Inc. 2026

Targeted HSGs: definition

NTT

A targeted HSG G for a class $\mathcal{C}$ and an input distribution $\mathcal{D}$

Syntax: $G(x)$ deterministically outputs a list $S := G(x)$.
Hitting guarantee: For each algorithm $A \in \mathcal{C}$, the list $S = G(x)$ satisfies the hitting condition for $A(x; \cdot)$ with high probability over $x \sim \mathcal{D}$.*

Known constructions [CT22; CT23]: A targeted HSG that

- works for a class of uniform algorithms and any polynomial-time samplable input distribution under (very) strong complexity assumptions.
- has output list size $|x|^{\epsilon}$ for a small constant $\epsilon > 0$.

*While a standard HSG outputs a single list that satisfies the hitting condition for $A(x; \cdot)$ for every x, a targeted HSG may output a list that depends on x.

© NTT, Inc. 2026

Targeted HSGs: an application

Chen, Rothblum, and Tell [CRT25] obtain **NIWI** from an injective OWF and a targeted HSG (without using public-key crypto).

Key idea (details later)

Use a targeted HSG to **instantiate Fiat–Shamir for MPC-in-the-head**, thereby obtaining non-interactive MPC-in-the-head in the plain model.
(No random oracle, CRS, or verifier message.)

© NTT, Inc. 2026

Targeted HSGs: an application

Chen, Rothblum, and Tell [CRT25] obtain **NIWI** from an injective OWF and a targeted HSG (without using public-key crypto).

Key idea (details later)

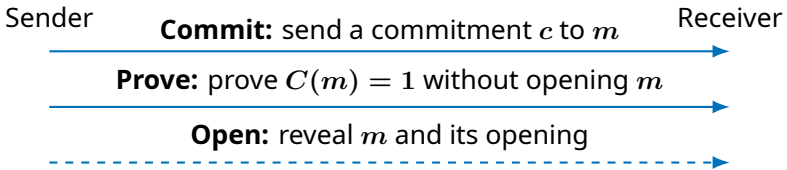
Use a targeted HSG to **instantiate Fiat–Shamir for MPC-in-the-head**, thereby obtaining non-interactive MPC-in-the-head in the plain model.
(No random oracle, CRS, or verifier message.)

Can non-interactive MPC-in-the-head give other cryptographic applications?

© NTT, Inc. 2026

Our application: commit-and-prove

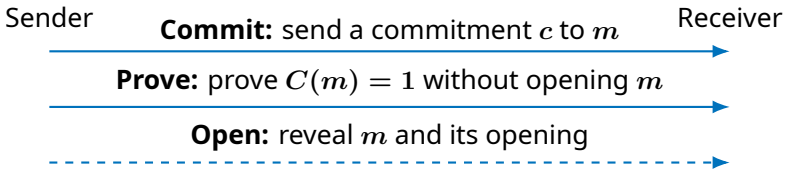
A commitment to a secret m, plus **Prove** for a public predicate C.



© NTT, Inc. 2026

Our application: commit-and-prove

A commitment to a secret m, plus **Prove** for a public predicate C.

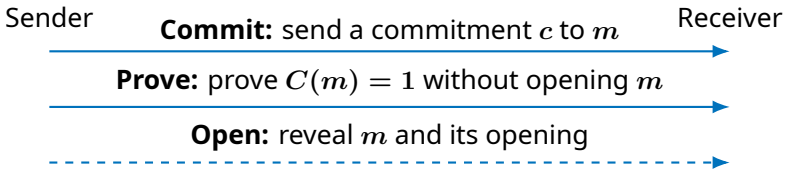


- **Soundness:** if the proof is accepted, an opening to a value satisfying C exists.
- **WI:** if $C(m_0) = C(m_1) = 1$, the entire **Commit + Prove** executions using m_0 and m_1 are indistinguishable.

© NTT, Inc. 2026

Our application: commit-and-prove

A commitment to a secret m, plus **Prove** for a public predicate C.



- **Soundness:** if the proof is accepted, an opening to a value satisfying C exists.
- **WI:** if $C(m_0) = C(m_1) = 1$, the entire **Commit + Prove** executions using m_0 and m_1 are indistinguishable.
- **Black-box (BB):** use the underlying crypto primitive only via its inputs and outputs.

Prior BB construction: interactive (three rounds) [KOS18]

© NTT, Inc. 2026

Our result

Theorem

A suitable non-interactive commitment and a sufficiently strong targeted HSG give **non-interactive black-box WI commit-and-prove**, with soundness against uniform PPT adversaries.

- The required commitment scheme has a BB construction from an injective OWF with efficiently recognizable range.
- The HSG is the same as in [CRT25].

© NTT, Inc. 2026

Our result

Theorem

A suitable non-interactive commitment and a sufficiently strong targeted HSG give **non-interactive black-box WI commit-and-prove**, with soundness against uniform PPT adversaries.

- Soundness is guaranteed only against **uniform PPT adversaries**. ☹
- Known instantiations of the targeted HSG require **very strong complexity assumptions**.* ☹
- **Non-interactive in the plain model** (i.e., no random oracle, CRS, or verifier message). ☺

*This footnote is too small to contain a formal description of the assumptions.

© NTT, Inc. 2026

Techniques

© NTT, Inc. 2026

Recall: MPC-in-the-head (MPCitH) [IKOS09]

NTT

Prv(x, w) proves $R(x, w) = 1$ to **Vrfy**(x).

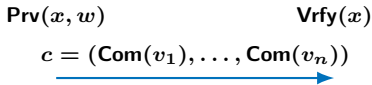
Prv(x, w) **Vrfy**(x)

© NTT, Inc. 2026

Recall: MPC-in-the-head (MPCitH) [IKOS09]

$\mathbf{Prv}(x, w)$ proves $R(x, w) = 1$ to $\mathbf{Vrfy}(x)$.

P1 Secret-share w into $w_1, \ldots, w_n$.
Run an MPC for $R(x, w)$ with party P_i holding w_i.
Commit to every party's view v_i.



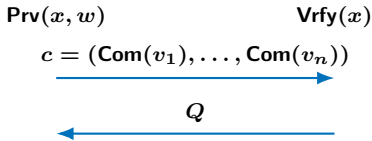
© NTT, Inc. 2026

Recall: MPC-in-the-head (MPCitH) [IKOS09]

$\mathbf{Prv}(x, w)$ proves $R(x, w) = 1$ to $\mathbf{Vrfy}(x)$.

P1 Secret-share w into $w_1, \ldots, w_n$.
Run an MPC for $R(x, w)$ with party P_i holding w_i.
Commit to every party's view v_i.

V1 Choose a random subset $Q \subseteq [n]$ of parties.



© NTT, Inc. 2026

Recall: MPC-in-the-head (MPCitH) [IKOS09]

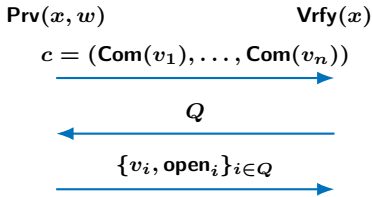
$\mathbf{Prv}(x, w)$ proves $R(x, w) = 1$ to $\mathbf{Vrfy}(x)$.

P1 Secret-share w into $w_1, \ldots, w_n$.
Run an MPC for $R(x, w)$ with party P_i holding w_i.
Commit to every party's view v_i.

V1 Choose a random subset $Q \subseteq [n]$ of parties.

P2 Open the views in Q, with opening information $\mathbf{open}_i$.

V2 Check outputs and consistency of the opened views.



© NTT, Inc. 2026

Recall: MPC-in-the-head (MPCitH) [IKOS09]

NTT

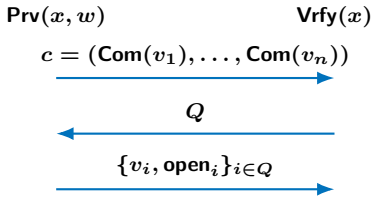
$\mathbf{Prv}(x, w)$ proves $R(x, w) = 1$ to $\mathbf{Vrfy}(x)$.

P1 Secret-share w into $w_1, \ldots, w_n$.
Run an MPC for $R(x, w)$ with party P_i holding w_i.
Commit to every party's view v_i.

V1 Choose a random subset $Q \subseteq [n]$ of parties.

P2 Open the views in Q, with opening information $\mathbf{open}_i$.

V2 Check outputs and consistency of the opened views.



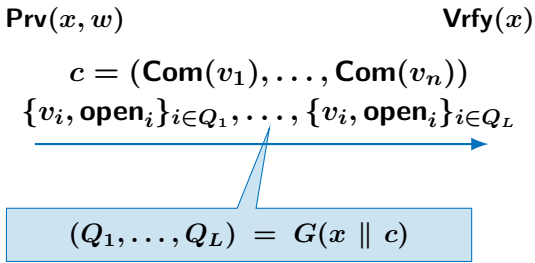
Soundness: by MPC robustness, cheating needs many inconsistencies, so a large Q catches one w.h.p.

Privacy: by MPC privacy, the opened views hide w if $|Q|$ is small enough.

© NTT, Inc. 2026

Recall: MPCitH + targeted HSGs [CRT25]

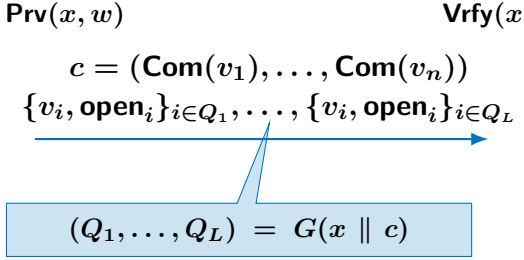
Generate the subsets Q using a **targeted HSG** G.



© NTT, Inc. 2026

Recall: MPCitH + targeted HSGs [CRT25]

Generate the subsets Q using a **targeted HSG** G.



Soundness

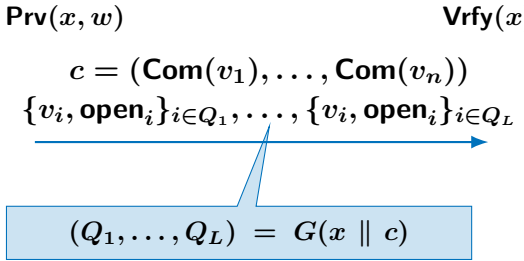
Follows from **hitting guarantee** of G.* (Key point: **Vrfy** is derandomized!)

*It suffices for G to be a targeted HSG for $\mathbf{DTIME}[N]^{\mathbf{SAT}}$, where N is its input length. In particular, hitting for a test that recovers the committed views and detects cheating using a SAT oracle suffices.

© NTT, Inc. 2026

Recall: MPCitH + targeted HSGs [CRT25]

Generate the subsets Q using a **targeted HSG** G.



Soundness

Follows from **hitting guarantee** of G.* (Key point: **Vrfy** is derandomized!)

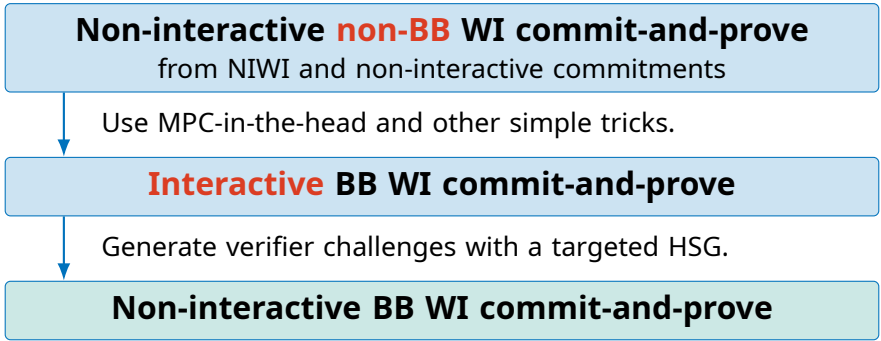
WI

Follows from **small list size** of G: since $L \ll n$, the total number of opened views can be kept below the MPC privacy threshold.

*It suffices for G to be a targeted HSG for $\mathbf{DTIME}[N]^{\mathbf{SAT}}$, where N is its input length. In particular, hitting for a test that recovers the committed views and detects cheating using a SAT oracle suffices.

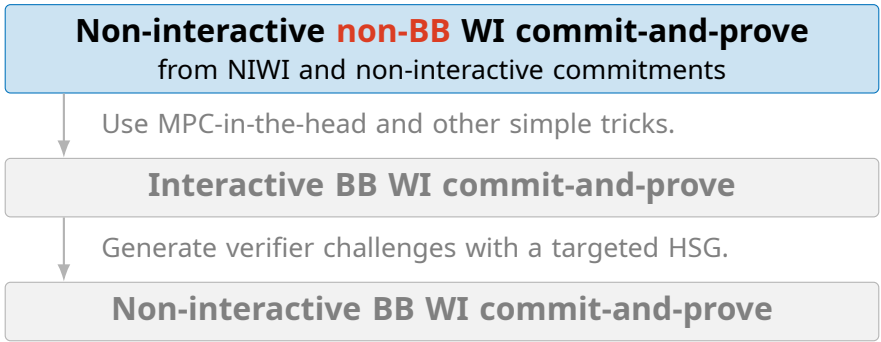
© NTT, Inc. 2026

Our approach



© NTT, Inc. 2026

Our approach

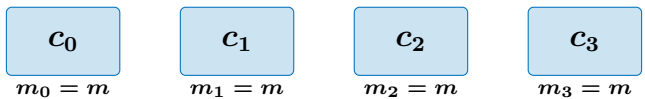


© NTT, Inc. 2026

Our non-BB construction

NTT

1. **Commit:** make four commitments $c_0, \ldots, c_3$ to the same m.
2. **Open:** open all four; the committed value is defined to be the **most frequent value**.* ← deliberately relaxed (not requiring all four to be equal)

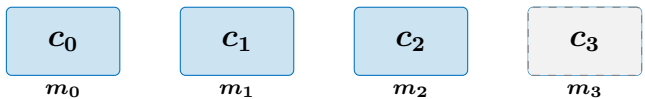


*Ties are broken in a canonical way.

© NTT, Inc. 2026

Our non-BB construction

1. **Commit:** make four commitments $c_0, \ldots, c_3$ to the same m.
2. **Open:** open all four; the committed value is defined to be the **most frequent value**.* ← deliberately relaxed (not requiring all four to be equal)
3. **Prove:** use NIWI to prove that three of the four commit to **at most two values satisfying** C. ← deliberately relaxed, again



$\exists i^*, m', m'' \; \forall i \in \{0,1,2,3\} \setminus \{i^*\}$:
c_i commits to $m_i \in \{m', m''\}$ s.t. $C(m_i) = 1$

*Ties are broken in a canonical way.

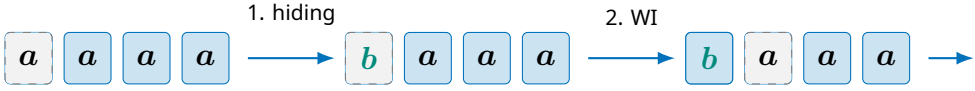
© NTT, Inc. 2026

Witness indistinguishability

Straightforward hybrid argument.

For $C(a) = C(b) = 1$, change an all-a execution into an all-b execution.

1. Use hiding to change the unused commitment from a to b.
2. Use WI to move the position unused in NIWI.
   (A mixture of a and b still satisfies the NIWI statement.*)



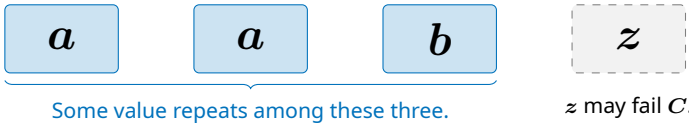
Gray: the position unused in NIWI.

*__NIWI statement:__ three of the four commit to at most two values satisfying C.

© NTT, Inc. 2026

Soundness

NIWI statement: three of the four commit to **at most two values satisfying C.**



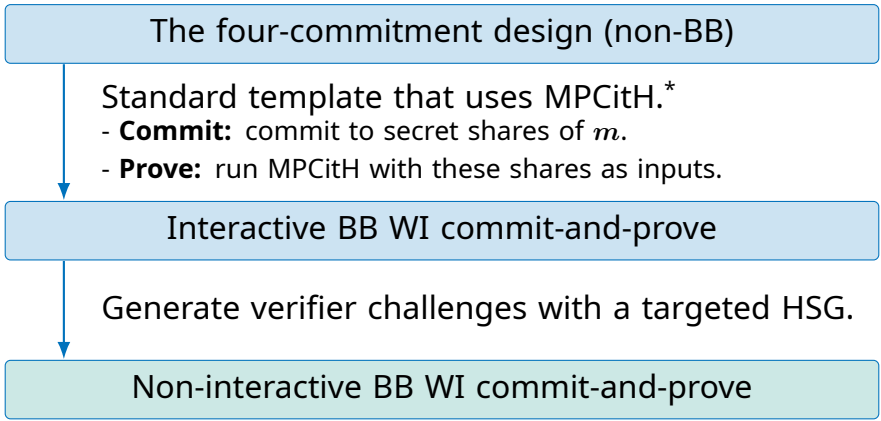
1. Some value satisfying C occurs **at least twice**.
2. A value failing C occurs only in the unused position, **at most once**.
3. Thus, the **most frequent value satisfies C**, and this value is defined as the committed value in the Open phase.

$\Rightarrow$ When NIWI is accepted, an opening to a value satisfying C exists![*]

[*]For simplicity, assume that even a malformed commitment has a unique value to which it can be opened.

© NTT, Inc. 2026

The final BB construction



*Other simple tricks are also needed (see the paper for details).

© NTT, Inc. 2026

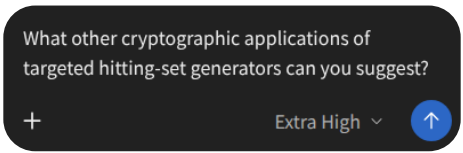
Conclusion

We give a new application of making MPC-in-the-head non-interactive using targeted HSGs:

Non-interactive BB WI commit-and-prove

In the plain model, against uniform PPT adversaries.

A natural open question: what other applications are possible?



© NTT, Inc. 2026

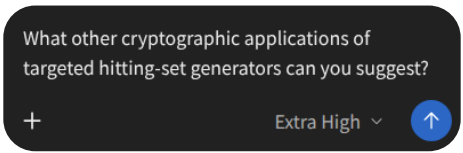
Conclusion

We give a new application of making MPC-in-the-head non-interactive using targeted HSGs:

Non-interactive BB WI commit-and-prove

In the plain model, against uniform PPT adversaries.

A natural open question: what other applications are possible?



Thanks!

© NTT, Inc. 2026

References

[CLOS02] Ran Canetti et al. “Universally composable two-party and multi-party secure computation”. In: *34th ACM STOC*. ACM Press, May 2002, pp. 494–503.

[CRT25] Lijie Chen, Ron D. Rothblum, and Roei Tell. “Fiat-Shamir in the Plain Model from Derandomization (Or: Do Efficient Algorithms Believe that NP = PSPACE?)” In: *57th ACM STOC*. Ed. by Michal Koucký and Nikhil Bansal. ACM Press, June 2025, pp. 977–985.

[CT22] Lijie Chen and Roei Tell. “Hardness vs Randomness, Revised: Uniform, Non-Black-Box, and Instance-Wise”. In: *62nd FOCS*. IEEE Computer Society Press, Feb. 2022, pp. 125–136.

[CT23] Lijie Chen and Roei Tell. “When Arthur Has Neither Random Coins Nor Time to Spare: Superfast Derandomization of Proof Systems”. In: *55th ACM STOC*. Ed. by Barna Saha and Rocco A. Servedio. ACM Press, June 2023, pp. 60–69.

[GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. “How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority”. In: *19th ACM STOC*. Ed. by Alfred Aho. ACM Press, May 1987, pp. 218–229.

[IKOS09] Yuval Ishai et al. “Zero-Knowledge Proofs from Secure Multiparty Computation”. In: *SIAM Journal on Computing* 39.3 (2009), pp. 1121–1152.

References

[KOS18] Dakshita Khurana, Rafail Ostrovsky, and Akshayaram Srinivasan. “Round Optimal Black-Box “Commit-and-Prove””. In: *TCC 2018, Part I*. Ed. by Amos Beimel and Stefan Dziembowski. Vol. 11239. LNCS. Springer, Cham, Nov. 2018, pp. 286–313.

Assumption behind the HSG (informal)

NTT

Assumption: non-batch-computability [CT22; CRT25]

There is f with k output bits such that

- each output bit of $f(x)$ is computable in time T, but
- printing all k bits in a batch (or even 99% of them) requires substantially more than T time, even with a SAT oracle,

for x from *any* polynomial-time samplable distribution.

Assumption behind the HSG (detailed)

NTT

Assumption: non-batch-computability [CRT25]

For a sufficiently small $\gamma > 0$ and $k = n^{\gamma}$, there is $f\colon \{0,1\}^n \to \{0,1\}^k$ such that

- each bit $f(x)_i$ is computable in time $T = \mathbf{poly}(n)$ (so all k bits in time kT), but
- for every polynomial-time samplable X, no $(T \cdot k^{0.01})$-time algorithm with a SAT oracle agrees with $f(x)$ on a 0.99 fraction of the k bits, except for a negligible fraction of $x \sim X_n$.

Consequence [CRT25]

A polynomial-time targeted HSG for $\mathbf{DTIME}[N]^{\mathbf{SAT}}$ and every polynomial-time samplable distribution, with negligible failure probability and list size $N^{O(\epsilon)}$ (where ϵ is proportional to γ).

Why commit-and-prove?

NTT

A tool for enforcing honest behavior [GMW87; CLOS02]

- Commit to the input and randomness, and later prove that each protocol message is consistent with the committed values.

Why black-box?

- Non-BB: often expensive and dependent on the implementation of the primitive (e.g., ZK proofs of its correct evaluation).
- BB: use the primitive only via its inputs and outputs, so the cost is independent of its implementation.

Prior work

NTT

Interactive & non-BB: straightforward

Commit to m, then use interactive ZK to prove "c commits to some m satisfying $C(m) = 1$."

Non-BB: the statement includes the commitment verification circuit.

Interactive & BB [KOS18]

Three-round WI commit-and-prove from injective OWFs.

Three rounds in total, counting both Commit and Prove.

Prior work

NTT

Interactive & non-BB: straightforward

Commit to m, then use interactive ZK to prove "c commits to some m satisfying $C(m) = 1$."

Non-BB: the statement includes the commitment verification circuit.

Interactive & BB [KOS18]

Three-round WI commit-and-prove from injective OWFs.

Three rounds in total, counting both Commit and Prove.

Our goal

Make **both Commit and Prove non-interactive**, while using the primitive in a BB way.

Failed attempts

NTT

Attempt 1: one commitment

Commit / Open: use a non-interactive commitment.

Prove: use NIWI to prove that c commits to an m satisfying C.

✘ **WI is not guaranteed.**
For fixed c, all witnesses contain the same m, so NIWI need not hide m.

Failed attempts

NTT

Attempt 1: one commitment

Commit / Open: use a non-interactive commitment.

Prove: use NIWI to prove that c commits to an m satisfying C.

✘ **WI is not guaranteed.**
For fixed c, all witnesses contain the same m, so NIWI need not hide m.

Attempt 2: two commitments

Commit: commit to the same m twice.
Open: open both to the same m.

Prove: use NIWI to prove that at least one commitment commits to a value satisfying C.

✘ **Soundness fails.**
The proof can pass even if the commitments commit to different values.